Report for the Semester Project: "Medialab Minesweeper"

Course: Multimedia Technology
Author: Aristotelis Grivas

The design of the game is based on the Element class, which is used to create the game elements (tiles). Each element is a rectangle. The constructor of an element sets its horizontal and vertical position (x, y) as well as whether it contains a bomb (hasBomb). Using this class, we can build the main game scene via the CreateGame method, which returns a Parent node to which we add the Elements as children. This method uses the following:

1. **getNeighbors() method:** Returns a list of neighboring elements for each element. Using this, we can set the text of each element by checking how many of its neighbors contain a bomb.
2. **minetable 2D array:** Contains information about whether a bomb exists at a given x, y coordinate (0 → no bomb, 1 → bomb, 2 → super bomb).

Based on the above, we set the text for each element (X for a bomb, a number or blank otherwise) and hide it until the user left-clicks it (calling the open() method of the Element class) or right-clicks it (calling the flagIt() method, which marks the tile in green).

The GameArray[][] array contains all elements added to the scene along with their respective positions.

For the other scenes, we use the FXML files sample1.fxml and sample2.fxml, whose functionality is controlled by SampleController and Sample2Controller, respectively.

- **sample1.fxml scene:** This is the initial scene shown when the program starts. It contains the main menu with functions described in the project instructions (Create, Load, Start, etc.), as well as messages about flagged elements, the number of bombs, and the remaining time. This scene is different from the main game scene described above, which appears when the StartGame button is pressed.
- **sample2.fxml scene:** This scene shows a popup window where game difficulty settings can be configured. It appears when CreateGame is pressed, calling the GiveNumbers method of SampleController. It consists of four input fields for time, difficulty, number of bombs, and number of super bombs. Values are submitted via the Submit Settings button. All buttons in this scene are controlled by Sample2Controller.

If the input values are out of bounds, exceptions are printed to the output, and the user must re-enter valid values. Once the variables are within allowed ranges, pressing SubmitSettings creates the file SCENARIO-ID.txt via the CreateFile() function of SampleController, writing each variable on a separate line as specified.

Next, pressing the LoadGame button calls the Load() method of SampleController, transferring the previously entered values to the main program variables. At this point, the game can start using the StartGame button, which calls the StartGame method and then the begin method of the main program. This opens the game scene with the corresponding number of elements and starts a countdown timer.

The SampleController also provides functionality for the Details, Solve, and ExitGame buttons:

- **Details:** Writes the results of each game to the details.txt file, each game on a new line. On each press, the last five lines of the file are read (the last five games, with the most recent appearing last).

Assumptions:

- Since the function of SCENARIO-ID was unclear, we assumed that this file is recreated with each new game in the medialab folder, writing new values each time. The application supports multiple scenarios, but the desired scenario must be specified each time. This avoids InvalidDescriptionException (since four numbers are always written on four lines). To test the exception, the file can be manually modified before loading or starting a game.
- details.txt and mines.txt are located in the medialab folder. mines.txt is recreated for each game, while details.txt is updated.
- The medialab folder is located in the project directory and contains the above files and any images used in the interface.
- Super bombs appear in the first four flagged tiles.

Non-implemented functionality:

- The Menu and MainGame are not in the same scene.

Additional Notes:

- To start a new game, the desired scenario must be loaded first; otherwise, the game will not start, and a message will be displayed.

- The methods StaticBufferSize and countLineJava are used to read the last five lines of details.txt for the Details function.
- Various messages were added for better communication with the user.
- If the player wins, loses, chooses to see the solution, or time runs out, the result (Defeat/Victory) is printed in the output and the scene is “frozen.”
- The application was developed in Eclipse IDE, in combination with SceneBuilder.